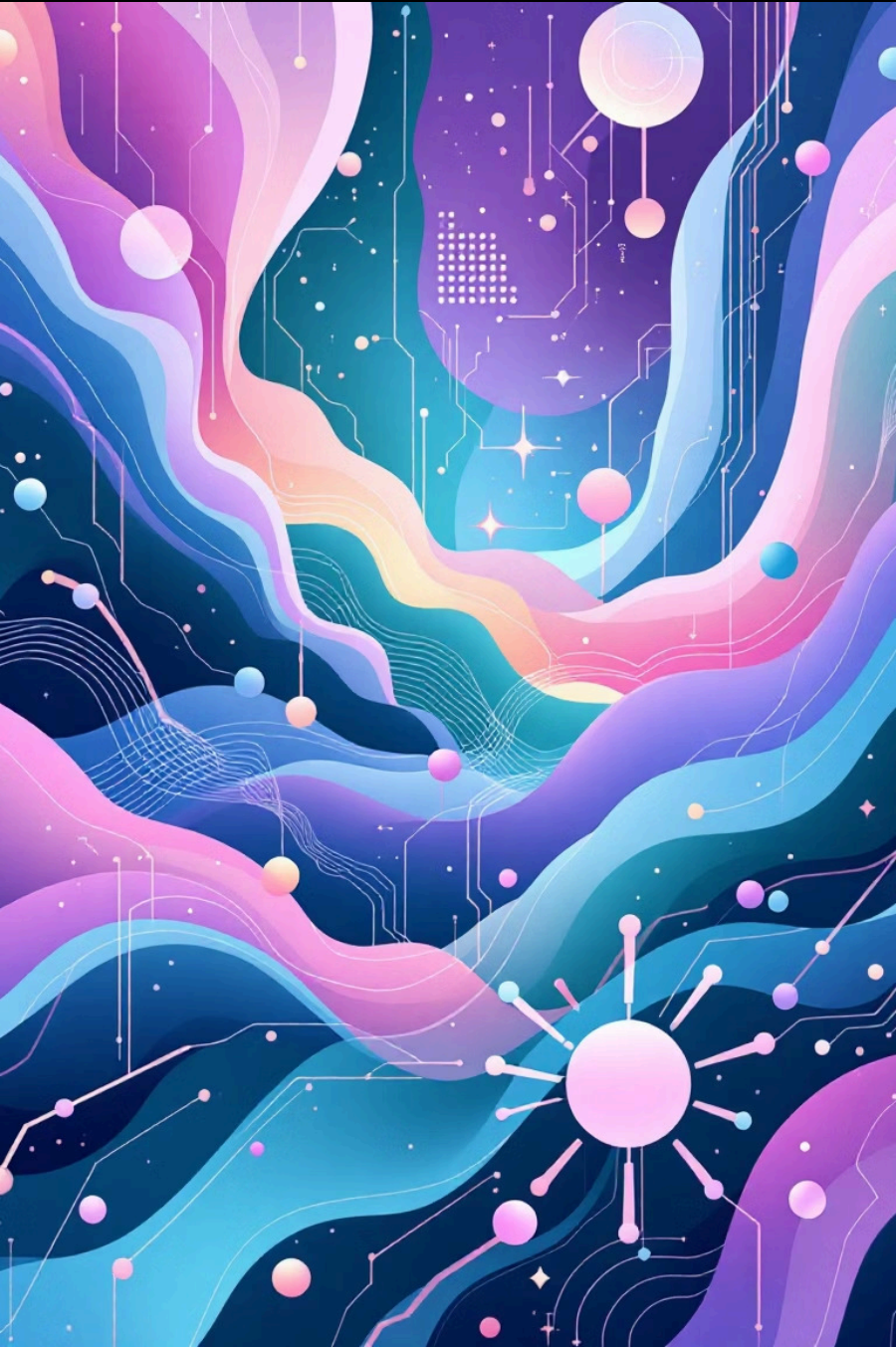




Day 11 — Embeddings + Vector DB

Semantic search with vectors • [CloudByteHub.ai](https://cloudbytehub.ai)

Core Concepts

Understanding the building blocks of modern semantic search requires three key components working together:



Embeddings

Transform text into high-dimensional vectors that capture semantic meaning. Words with similar meanings cluster together in vector space.



Vector Database

Efficiently stores and retrieves vectors using nearest neighbor algorithms. Enables lightning-fast similarity searches across millions of vectors.



Use Cases

Powers RAG (Retrieval-Augmented Generation) systems and semantic search. Find relevant content based on meaning, not just keywords.

Toy Demo Flow

Let's walk through a simplified implementation using FAISS basics. This hands-on approach demonstrates core principles without production complexity:

01

Hash Embeddings (Offline)

Pre-compute vector representations of your text corpus. This one-time process converts documents into numerical embeddings stored for quick retrieval.

03

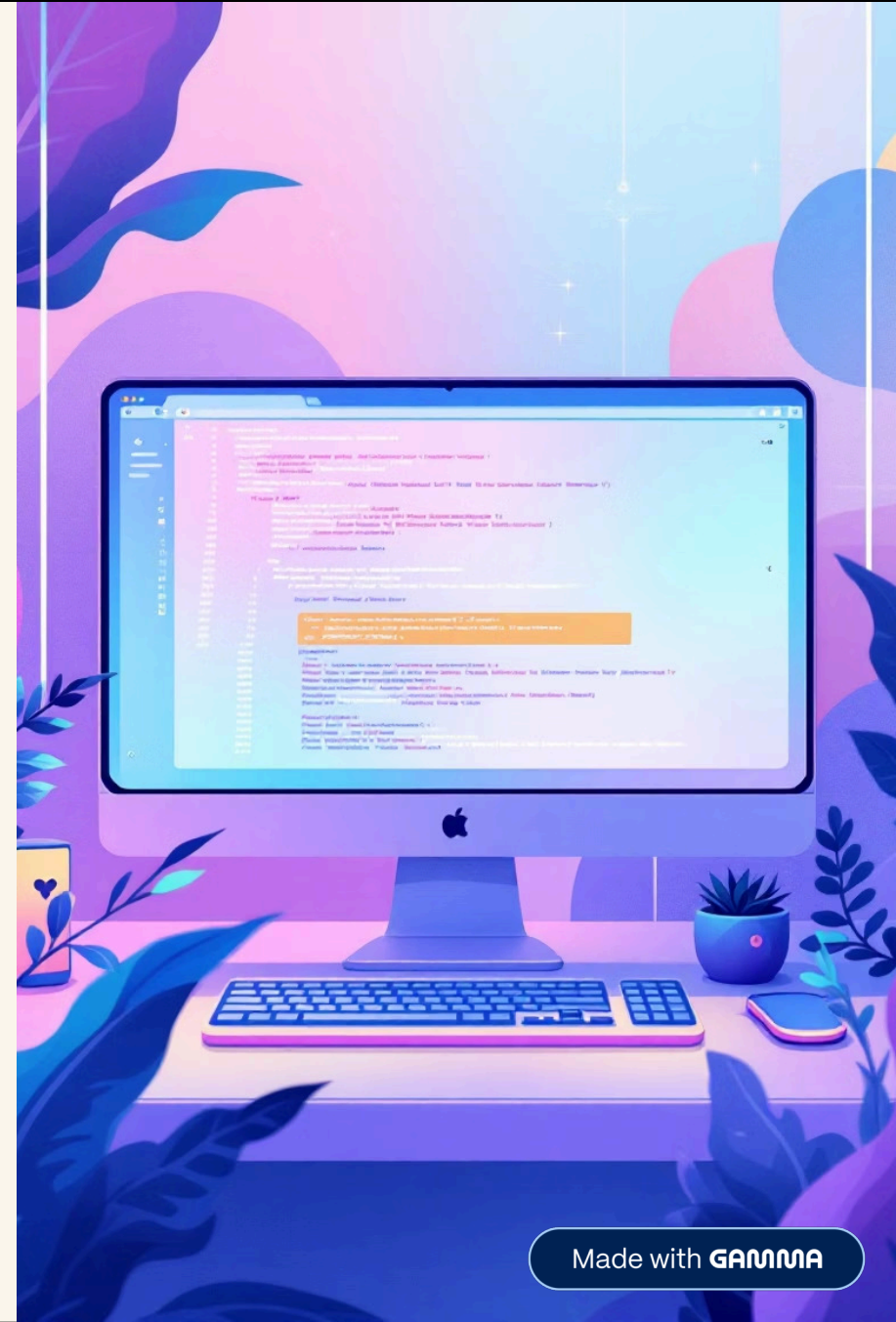
Cosine Similarity Search

Query the index to find top-k nearest neighbors. Cosine similarity measures the angle between vectors, returning the most semantically relevant results.

02

Build In-Memory Index

Create a searchable index structure using numpy arrays. The index organizes vectors to enable fast similarity comparisons without complex dependencies.



Important Notes & Safety

Technical Details

- **Dependency-free demo:** No FAISS installation required—pure Python implementation
- **Educational focus:** Designed for learning, not production deployment
- **Scalability limits:** In-memory approach works for small datasets only

Project Information

- **License:** MIT License for free use and modification
- **Source:** Independent personal project by CloudByteHub.ai
- **Purpose:** Hands-on exploration of vector search fundamentals

